Übungsblatt 9: Multicore & Finale

Mikrocomputertechnik 3 · Zustände, Atomics und Systemrechnungen auf Papier

Musterlösung

Ziel und Arbeitsweise

Sechs Pflichtaufgaben, vollständig auf Papier lösbar. Geben Sie Modellannahmen, Zwischenwerte und Einheiten an. Es sind keine Hardwaremessungen oder neue Prüfungsformalien enthalten.

Bearbeitungszeit: etwa 75–90 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht: Aufgaben 1–2 (10 min), Aufgabe 3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Die Vorlesungsphasen enthalten Lösungsbesprechungen.

MESI bedeutet Modified, Exclusive, Shared, Invalid. Hart ist eine Hardware-Ausführungsinstanz, Thread ein Softwareablauf. RMW heißt Read-Modify-Write, Lesen, Ändern und Schreiben. LR/SC steht für Load-Reserved/Store-Conditional. KiB = 1024 Byte, MHz = 10^6 Takte je Sekunde.

Aufgabe 1: MESI mit Datenwerten (6 Punkte)

Lehrmodell K: zwei Datencaches, Write-Back, Write-Allocate, stabile MESI-Zustände. Jede Zeile umfasst 64 Byte. L ist die untere kohärente Speicherebene; anfangs $x = 0$, beide Caches I. Beim Fremdread auf M werden die aktuellen Daten im Modell nach L übertragen und beide Kopien S. Geänderte Daten werden vor M-Auswurf zurückgeschrieben.

1. Ergänzen Sie Zustände mit Werten und den unteren Stand nach jedem abgeschlossenen Schritt. (3 P)
2. Warum darf L nach dem ersten Write noch null sein? Darf Kern 1 beim folgenden Read trotzdem null liefern? (1 P)
3. Welche Aktionen müssen beim Write auf S vor dem stabilen Abschluss erfolgen? (1 P)
4. Separater Fall: Beide Caches halten S:5, dann wirft Kern 0 seine saubere Kopie aus. Muss Kern 1 automatisch E werden? Begründen Sie. (1 P)

Aktion	Kern 0	Kern 1	L
Start	I	I	0
Kern 0 liest x			
Kern 1 liest x			
Kern 0 schreibt $x = 5$			
Kern 1 liest x			
Kern 1 schreibt $x = 7$			
Kern 1 wirft seine Zeile aus			
Kern 0 liest x			

Musterlösung 1

Aktion	Kern 0	Kern 1	L
Start	I	I	0
Read Kern 0	E:0	I	0
Read Kern 1	S:0	S:0	0
Write Kern 0: 5	M:5	I	0
Read Kern 1	S:5	S:5	5
Write Kern 1: 7	I	M:7	5
Auswurf Kern 1	I	I	7
Read Kern 0	E:7	I	7

1. Nur M enthält einen gegenüber L geänderten Wert. Exklusive M-/E-Kopien schließen andere gültige Kopien aus, gemeinsame S-Werte stimmen überein. (3 P)
2. Write-Back hält die Änderung zunächst im privaten Besitzer. Der abgeschlossene fremde Read muss fünf über einen aktuellen Pfad erhalten, nicht die alte Null von L oder einer I-Kopie. (1 P)
3. Schreibrecht übernehmen, fremde Gültigkeit entziehen und die nötigen Bestätigungen abwarten. Die abgesendete Anfrage allein genügt nicht. (1 P)
4. Nein. Eine verbleibende S-Kopie kann S bleiben, weil der Controller nicht automatisch die vollständige Abwesenheit weiterer Kopien weiß. (1 P)

Aufgabe 2: False Sharing und Besitzerwechsel (4 Punkte)

Cachezeile 64 Byte, Objektgröße je 4 Byte, ausgerichtete Basis `0x20001000`. Kern 0 schreibt nur Objekt a bei Offset 0, Kern 1 nur Objekt b bei Offset 4. Die Objekte sind softwareseitig getrennt zugeteilt.

1. Geben Sie beide Adressen und den vollständigen gemeinsamen Zeilenbereich an. (1 P)
2. Anfangs besitzt Kern 0 die Zeile bereits M. Schreibfolge: Kern 0, Kern 0, Kern 1, Kern 1, Kern 0. Wie oft wechselt die exklusive Zuständigkeit zwischen Kernen? Warum nicht bei jedem Write? (1 P)
3. b wird nun bei Offset 64 abgelegt. Bestimmen Sie die neue Adresse und erklären Sie, weshalb bloße Basisausrichtung bei Offset 4 nicht genügt. (1 P)
4. Würde solcher Abstand ein verlorenes Update zweier Schreiber auf denselben ungeschützten Zähler reparieren? (1 P)

Musterlösung 2

1. a: `0x20001000`, b: `0x20001004`. Gemeinsame Zeile `0x20001000` bis einschließlich `0x2000103F`. (1 P)
2. Besitzerfolge $0, 0, 1, 1, 0$ mit anfangs Besitzer null: zwei Wechsel, beim ersten Write von eins und danach beim Write von null. Weitere lokale Writes auf eigener M-Zeile benötigen keinen erneuten Wechsel. (1 P)
3. Offset vierundsechzig ergibt `0x20001040`, nächste Zeile. Basis und Stride, Elementabstand, müssen zusammen zur Zeilengröße passen. (1 P)
4. Nein. False Sharing betrifft verschiedene Objekte derselben Zeile. Das Update derselben Variable benötigt weiterhin atomare RMW oder geeignete Sperre. Padding kostet zudem Speicher und kann Lokalität verschlechtern. (1 P)

Aufgabe 3: Lost Update und LR/SC (6 Punkte)

1. Im Hardwaremodell lesen beide Harts Zähler null, erhöhen ihren lokalen Registerwert und schreiben zurück. Notieren Sie eine vollständige Schrittfolge, den Endwert und warum MESI ihn nicht korrigiert. (2 P)
2. Was bedeutet derselbe Konflikt bei einem gewöhnlichen ungeschützten C-`int`? Darf man die Endwerte eins oder zwei als garantierte C11-Liste angeben? Würden getrennte atomare Loads und Stores die Gruppenwirkung schon liefern? (1 P)
3. Binäres Schloss: frei null, belegt eins. Beide LR lesen null. SC von Hart 0 versucht eins und gewinnt. Danach versucht SC von Hart 1 ebenfalls eins und scheitert. Füllen Sie den Ablauf einschließlich Rückgabestatus aus. (2 P)
4. Darf Hart 1 nach einem neuen LR mit Wert eins den kritischen Abschnitt betreten? Muss jeder SC-Fehler einen fremden Store beweisen? (1 P)

Ereignis	gelesener Wert / Status	Schloss danach
LR Hart 0		
LR Hart 1		
SC Hart 0 mit eins, Erfolg		
SC Hart 1 mit eins, Fehler		
Neuer LR Hart 1		
Hart 0 gibt frei		

Musterlösung 3

1. Load Hart 0: null, Load Hart 1: null, Add Hart 0: eins, Add Hart 1: eins, Store Hart 0: eins, Store Hart 1: eins. Endwert eins. Kohärenz ordnet einzelne Zugriffe, nicht die getrennte RMW-Gruppe. (2 P)
2. Gewöhnliche konfliktierende C-Zugriffe ohne Synchronisation haben einen Data Race und undefiniertes Verhalten. Es gibt keine garantierte Liste dieser Endwerte. Atomare Einzel-Loads und -Stores können ebenfalls ein logisches Update verlieren. (1 P)

Ereignis	gelesener Wert / Status	Schloss danach
LR Hart 0	Wert 0	0
LR Hart 1	Wert 0	0
SC Hart 0 mit eins, Erfolg	Status 0	1
SC Hart 1 mit eins, Fehler	Status ungleich 0	1
Neuer LR Hart 1	Wert 1	1
Hart 0 gibt frei	Store 0	0

3. Die erfolgreiche Übernahme schreibt eins, der fehlgeschlagene SC schreibt nicht. Rückgabestatus und gelesener Wert sind unterschiedliche Größen. (2 P)
4. Nein, der belegt gelesene Wert verlangt Warten, keinen SC mit zwei. SC kann auch ohne fremden Store scheitern; jeder SC beendet seine Reservation. (1 P)

Aufgabe 4: Ordnung und Fortschritt (6 Punkte)

1. Ein Produzent schreibt einmalig gewöhnliche Nutzdaten `data = 42` und danach atomar `ready = 1` mit Release. Ein Konsument liest genau diese Eins mit Acquire und danach `data`. Nennen Sie Ergebnis und nötige Voraussetzungen. (2 P)
2. Separates wohldefiniertes Gegenmodell: Sowohl `data` als auch `ready` sind atomare Objekte, alle Zugriffe relaxed. Darf der Konsument bei ready eins noch data null beobachten? Warum wäre gewöhnliche ungeschützte Payload ein anderer Fall? (1 P)
3. Kern 0 hält Lock A und wartet auf B, Kern 1 hält B und wartet auf A. Skizzieren Sie den Wartekreis und eine gemeinsame Reihenfolge, die genau diesen Kreis verhindert. Dürfte ein Timeout ein fremdes Lock löschen? (2 P)
4. Auf demselben Hart hält ein Kontext L und wird unterbrochen. Die ISR spinnt auf L. Warum hilft eine weitere atomare Übernahmeoperation nicht? Hält eine lokale IRQ-Sperre andere Harts fern? (1 P)

ISR bedeutet Interrupt Service Routine, IRQ Interrupt Request. Acquire und Release sind Übernahme- und Freigabeordnung.

Musterlösung 4

1. Der spätere Datenread liefert zweiundvierzig. Erforderlich sind die passenden Zugriffe auf dasselbe Bereitschaftsflag: Acquire liest den zugehörigen Release-Wert, die Nutzdaten werden vorher geschrieben und später nicht ohne neue Disziplin überschrieben. (2 P)
2. Ja, die atomaren Einzelobjekte haben noch keine objektübergreifende Synchronisation. Mit gewöhnlicher konfliktierender Payload wäre der Zugriff dagegen ein C-Data-Race und undefiniert, keine solche erlaubte Ergebnisbehauptung. (1 P)
3. $0 \to B \to 1 \to A \to 0$ als Wartebeziehung. Alle übernehmen A vor B; damit kann kein B-Halter A erst nachträglich verlangen. (1 P) Ein Timeout darf eine vom Besitzer noch genutzte Sperre nicht einfach löschen. Dafür wäre eine gesonderte Abbruch- und Konsistenzdisziplin nötig. (1 P)
4. Der Besitzer kann erst nach ISR-Rückkehr freigeben, die ISR wartet gerade darauf. Mehr Atomizität beendet diesen Selbstdeadlock nicht. Lokale IRQ-Sperre betrifft nur den betreffenden Hart, keine anderen Harts. (1 P)

Aufgabe 5: Speedup, Effizienz und Verteilung (4 Punkte)

Feste Aufgabe: Ein-Kern-Zeit 100 Zeiteinheiten, serieller Anteil $s = 0{,}2$, paralleler Anteil 0,8. Zunächst perfekte Verteilung auf gleiche Kerne, kein Zusatzaufwand. $S(N) = 1/(s + (1-s)/N)$, $E(N) = S(N)/N$.

1. Berechnen Sie Zeit, Speedup und Effizienz bei vier Kernen. (1 P)
2. Berechnen Sie Speedup und Effizienz bei hundert Kernen sowie den Grenzfaktor. Wieso kann der Speedup nahe am Grenzwert und die Effizienz klein sein? (1 P)
3. Bei vier Kernen fallen nun fünf zusätzliche Zeiteinheiten an. Berechnen Sie neue Zeit und Speedup. Welchen normierten Wert hat $h(4)$ in der erweiterten Formel? (1 P)
4. Separat ohne diesen Zusatzaufwand: Die achtzig parallelen Einheiten sind als $10, 10, 10, 50$ verteilt. Bestimmen Sie parallele Abschlusszeit, Gesamtzeit und Speedup. (1 P)

Musterlösung 5

1. $T(4) = 20 + 80/4 = 40$, $S(4) = 100/40 = 2{,}5$, $E(4) = 2{,}5/4 = 62{,}5\,\%$. (1 P)
2. $T(100) = 20{,}8$, $S(100) = 100/20{,}8 \approx 4{,}8077$, $E(100) \approx 4{,}8077\,\%$. Grenze $1/s = 5$. Weitere Kerne verkürzen nur den verbleibenden parallelen Anteil. (1 P)
3. $T = 20 + 20 + 5 = 45$, $S = 100/45 \approx 2{,}2222$. $h(4) = 5/100 = 0{,}05$, also relativ zur ursprünglichen Ein-Kern-Zeit. (1 P)
4. Paralleler Abschluss nach $\max(10, 10, 10, 50) = 50$, Gesamtzeit $20 + 50 = 70$, Speedup $100/70 \approx 1{,}4286$. Dieselbe Gesamtarbeit ist nicht automatisch gleich verteilt. (1 P)

Aufgabe 6: Systemrechnungen zum Abschluss (6 Punkte)

1. Direkt abgebildeter Cache: 4 KiB Nutzdaten, 64 Byte je Zeile, 32-Bit-Byteadresse. Bestimmen Sie Satzanzahl, Offset-, Index- und Tagbits. Zerlegen Sie `0x20001084` und geben Sie die Zeilenbasis an. (2 P)
2. Hitzeit zwei Takte, Hitrate 0,9, zusätzliche Misspenalty vierzig Takte. Bestimmen Sie AMAT und die Zeit bei 100 MHz. (1 P)
3. Lehrtimer: Eingang 1 MHz, Teiler zehn, Zähler null bis neunhundertneunundneunzig. High bei null bis zweihundertneunundvierzig. Bestimmen Sie PWM-Frequenz, Periodendauer, aktive Takte und Duty Cycle. Warum darf ein Endregisterwert nicht ungeprüft als Periodenlänge verwendet werden? (2 P)
4. Ein DMA-Gerät schreibt ein Sensorfenster in einen Puffer. CPU-Caches sind untereinander kohärent; der Gerätevertrag ist noch offen. Darf die CPU daraus automatisch DMA-Kohärenz und ein fertiges Fenster folgern? Nennen Sie die gesondert nötigen Prüfungen. (1 P)

AMAT heißt Average Memory Access Time, PWM Pulse Width Modulation, DMA Direct Memory Access, CPU Central Processing Unit.

Musterlösung 6

1. $4096/64 = 64$ Sätze, Offset sechs Bit, Index sechs Bit, Tag $32 - 6 - 6 = 20$ Bit. (1 P) Adresse: Offset vier, Index zwei, Tag `0x20001`, Basis `0x20001080`. (1 P)
2. $\text{AMAT} = 2 + (1 - 0{,}9) \cdot 40 = 6$ Takte. Bei hundert MHz ist ein Takt zehn ns, also sechzig ns. (1 P)
3. Tausend Zählerwerte je Periode. $f = 10^6/(10 \cdot 1000) = 100$ Hz, $T = 1/f = 10$ ms, 250 aktive Takte, $D = 250/1000 = 25\,\%$. (1 P) Der inklusive Bereich null bis neunhundertneunundneunzig hat tausend Werte. Teilerkodierung, inklusives Ende und Vergleichsbedingung sind plattformspezifisch. (1 P)
4. Nein. Teilnehmerdomäne, Pufferzuständigkeit, Zugriffsbreite, Abschlussmeldung und Ordnung müssen festgelegt sein. Bei nichtkohärentem Gerätetransfer kommt passende Cachemaintenance hinzu. (1 P)

Optional: korrekte Hostbeispiele ausführen

Die Datei `src/lab09_atomics.c` nutzt C11-Atomics und POSIX Threads. Vom Repository-Hauptordner aus:

```
gcc -std=c11 -O2 -Wall -Wextra -Werror -pedantic -pthread \
    labs/src/lab09_atomics.c -o /tmp/lab09_atomics
/tmp/lab09_atomics
```

Erwartete Ausgabe:

```
Teilsummen: 10 26, gesamt 36
Getrennte atomare Zugriffe: Endwert 1
Atomare RMW: 40000
Geschuetzter Zaehler: 40000
Veroeffentlichte Daten: 42
Alle Prueffaelle bestanden.
```

1. Finden Sie die getrennt zugeteilten Ergebnisplätze und den Abschluss durch `pthread_join`.
2. Warum ist die serielle Lost-Update-Nachbildung wohldefiniert, ein ausgeführtes gewöhnliches racy C-`int`-Experiment aber kein Beweis?
3. Begründen Sie, warum relaxed für den reinen Zähler genügt, die Datenveröffentlichung dagegen Release/Acquire verwendet.
4. Optional: mit `-O1 -g -fsanitize=address,undefined` prüfen. Ein solcher Lauf beweist weder Fairness noch MESI-Geschwindigkeit.

Hinweise zur optionalen Aufgabe

Die Teilsummen werden in unterschiedliche Ergebnisobjekte geschrieben; erst nach beiden Joins liest der Hauptthread sie. Die verlorene Änderung wird mit zwei atomaren Loads auf null und anschließend zwei atomaren Stores von eins seriell vorgegeben. Dies prüft die Updatezerlegung ohne undefiniertes konkurrierendes C-Verhalten.

Je zwei tatsächlich gestartete Worker führen zwanzigtausend Inkremente aus. Die explizite atomare Additionsoperation erzeugt eine RMW; der Locklauf schützt eine gewöhnliche Variable über `atomic_flag`. Die einmalige Publikation schreibt Payload vor dem Release-Flag, der Konsument liest es mit Acquire und danach die Nutzdaten. Es gibt kein nachträgliches Überschreiben derselben Payload.

Die erzeugten Threads werden auch bei fehlgeschlagener zweiter Erzeugung abgewartet. Das Programm liest bei einem Threadfehler keine ungeklärten Ergebnisse. Es ist ein funktionaler Hostversuch, kein RISC-V-, ISR- oder Kohärenz-Zeitmodell.